

Creation of ml and other common instructions

Slide set 1

Nivriti Raisingh

Department of Information Engineering and Computer Science
University of Trento

October 6, 2025

1. Conversion of code into assembly

2. Instructions

- 2.1 Floating point instructions
- 2.2 MAC
- 2.3 RELU
- 2.4 Step function
- 2.5 Compare instructions
- 2.6 Other possible instructions
- 2.7 Tanh
- 2.8 LUI

3. New scheme

Conversion of code into assembly

- Used the RISC-V gcc 8.2.0 compiler on Compiler explorer to convert the provided C code into assembly
- Changed some instructions to reduce use the use of the stack in case of memory issues
- Made the following deductions:
 - Need to extend the current risc-v model to include floating point operations and add a register to hold floats
 - Need to figure out how to printf the result (can use the existing display mechanism or add a VGA interface)
 - Need to figure how to load the input arrays and weights

Floating point instructions

- A block containing 32 registers can be placed in the instruction decode, in parallel to the current register file. It is accessed when floats or doubles are needed
- The alu can be extended to have floating point operations or a separate block can be used (since the placement of bits is different)
- Load and store floats/doubles?

```
accumulator += weights[i] * inputs[i];
```



```
mul    r1,r1,r2  
add    rd,rd,r1
```



```
mac     rd,r1,r2  
fmac    frd,fr1,fr2
```

- Need to add an input to the register file that takes the destination address and gives out the corresponding value.
- operation: $rd = rd + r1 \times r2$
where rd: accumulator,
r1:weight[i], r2:x[i]
- opcode: 1010111
- func3: 001

RELU

```
hidden[i] = hidden[i] >= 0 ? hidden[i] : 0;
```



```
blt    a1,x0,.L1
add    a1,x0,x0
.L1:
```



```
relu   rd,r1
```

Finds if the value is positive, if it is not, it assigned 0, else the value stays the same operation: $rd = r1 \geq 0 ? r1 : 0$ where rd: destination address, r1: source register,

- opcode: 1010111
- funct3: 010

Step function



```
choice = output >= 0 ? 1 : 0;
```



```
step    rd,r1,r2  
stepi   rd,r1,imm
```

Finds if the value is positive, if it is not, it is assigned 0, else the value stays the same operation: $rd = r1 \geq r2 ? 1 : 0$

- opcode: 1010111 for **step** and 0010011 for **stepi**
- funct3: 100 for both **step** and **stepi**

Alternative: Compare instructions



Finds if the value in register 1 is greater or less than the value in register 2, rd=1 if it is, else rd=0

operation: $rd = r1 \text{ (comp) } r2 ? 1 : 0$

- opcode: 1010111
- funct3: same as the funct as branch operations

Other instructions



- **lyr** performs leaky relu. If $r1 \geq 0$, $rd=r1$, else $rd=r1 \times r2$.
 - opcode:1010111
 - funct3:011
- **max** finds the maximum between 2 numbers $r1$ and $r2$ and outputs the larger number
 - opcode:1010111
 - funct3:101
- **pos** finds if the number is positive or negative
 - opcode:1010111
 - funct3:110

Tanh?

Can be implemented using a lookup table or certain libraries (need to check how much time the computations take)

Load upper immediate

Not included in the current instruction set. Needs to be added to load large numbers.

Possible implementations:

- Lui would be part of the I-type instruction. The upper bits are shifted to the left before assigning it to the register.
- Instead of adding another instruction, we can add the upper bits, shift the value to the left and add the remaining bits

New scheme of instruction decode and execution stages

